



ASSESSMENT DECLARATION COVER SHEET

Please complete this cover sheet for each assessment submission. For group assessments, each group member must individually complete and submit a cover sheet.

STUDENT (REGISTRATION) NUMBER: 40735842

MODULE TITLE: SET09103 2025-6 TR1 001 - Advanced Web Technologies

DATE OF SUBMISSION: 12/12/2025

I declare that, except where explicitly acknowledged*, this assessment is my own work and has not been submitted for any other module or degree programme at Edinburgh Napier University or any other institution. This declaration is made in compliance with Edinburgh Napier University's [Academic Integrity Regulations](#).

***IMPORTANT:** Contributions from other sources include incorrectly cited quotations or content generated by Generative Artificial Intelligence (Gen AI) tools, such as ChatGPT. See [Artificial Intelligence Tools and Your Learning](#) for more information.

Note: Unless stated in the Assessment Brief, use of Grammarly for checking spelling and grammar is allowable in this assessment, but it cannot be used to generate content.

Please declare here your use of such tools in this assessment submission:

(Select one of the two options below)

☐ Yes, I have used Gen AI tools for this submission and I have described how below.

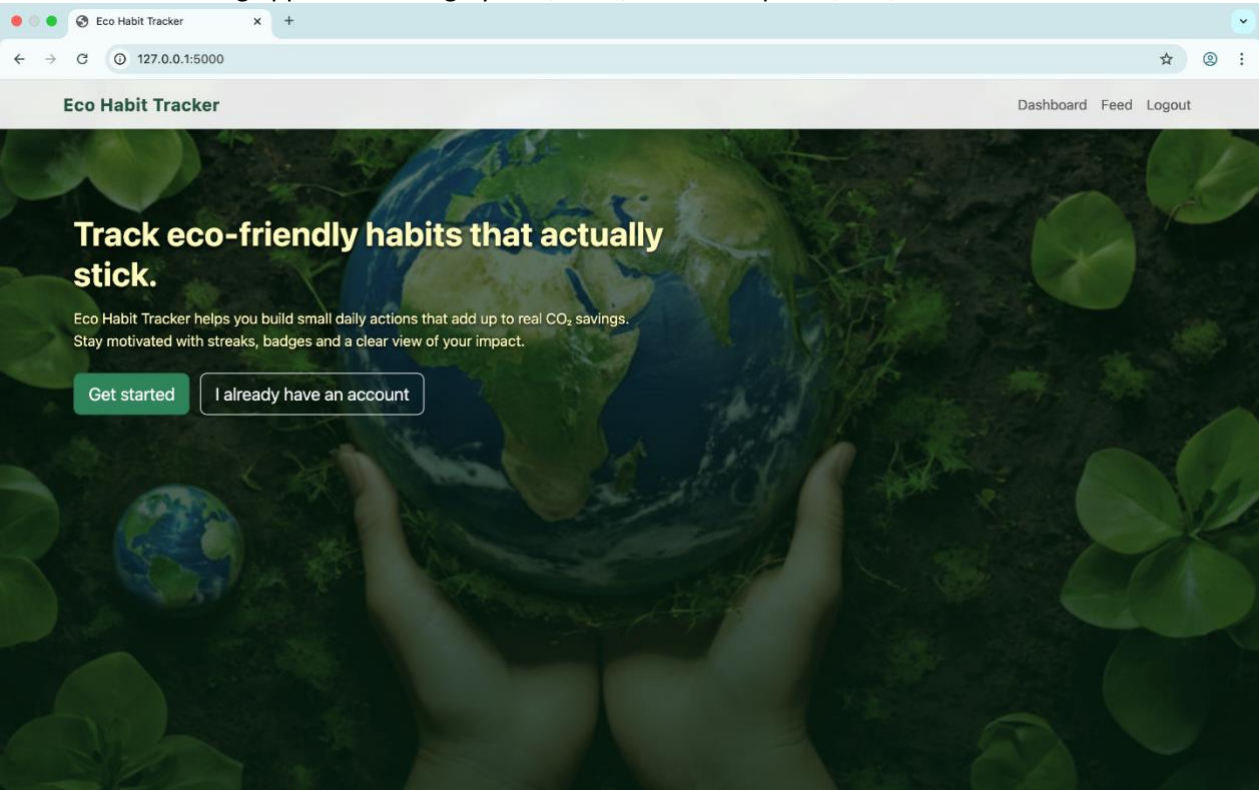
☒ No, I have not used Gen AI tools for this submission.

If you selected **Yes**, briefly describe (in less than 100 words) how you used these tools:

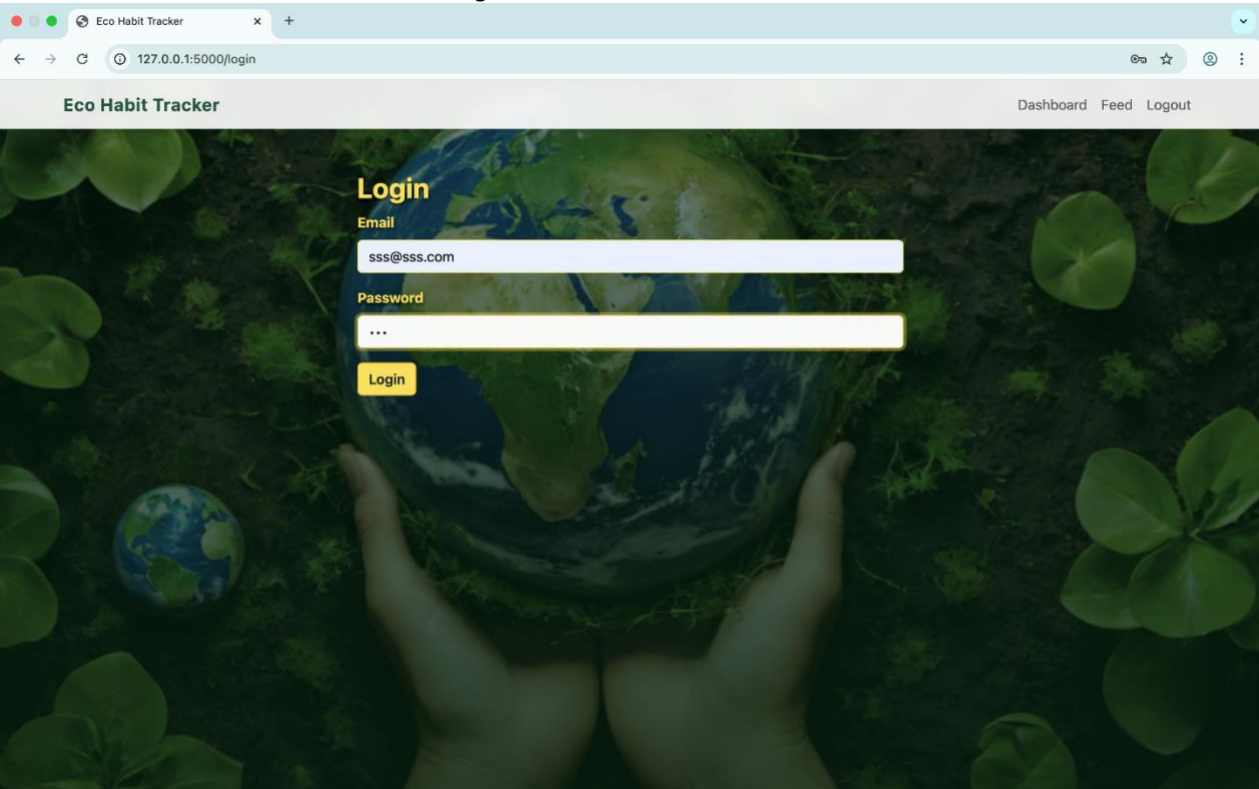
Click or tap here to enter text.

1.Introduction

For this coursework I built **Eco Habit Tracker**, a Flask web app that helps people see the impact of everyday sustainable choices. The idea is straightforward: users record small eco-friendly actions, watch them add up on a personal dashboard, and get a gentle sense of community from a shared activity feed. In **Part 1**, I focused on the design. I looked at existing applications like EcoHero and Habitica and defined my target audience, identified the main features, sketched wireframes of key pages, and proposed an ER diagram with tables needed for users, habits, logs, and eco badges. Now, in Part 2, I will take that design and create a working application using Python, Flask, HTML templates, CSS, and SQLite.



-Fig 1



-Fig 2

2. 2. Comparison of Plan and Final Implementation

2.1 Overview of core features

In the Part 1 design report I broke the project down into six main feature areas:

1. User accounts
2. Personal dashboard
3. Habit management
4. Community feed
5. Eco-badges
6. Time-limited group challenges

The last version has the first five features operational: you can sign up, log in, create new eco-friendly habits, track your progress, and see a dashboard with estimates of your CO₂ savings and streaks. There is also a badge system for longer streaks and community feed showing recent activity. Group challenges are the only major component that was left out this time.

2.2 Feature-by-feature comparison

User accounts

Planned: In Part 1 I argued that users needed private accounts so they could track their progress safely over time. The idea was to use basic email-and-password login and keep the main parts of the site the dashboard, community feed, and badges available only to authenticated users.

Implemented: Implemented: In Part 2 I set up registration and login endpoints using a User table within SQLite. Passwords are saved as encrypted hashes of plain text and Flask-Login handles user session management. The homepage is accessible to all. Any unauthenticated user attempting to reach secured routes such as /dashboard or /habit/<id> will be sent to the login page. After logging in users are immediately directed to their dashboard where they can view only the habits and logs linked to their account.

Differences: The authentication system is kept deliberately simple. There is no email verification, password reset, or account deletion. For now, the idea is to keep the workflow simple and focused without adding extra steps that are not necessary at this stage of the project.

Dashboard

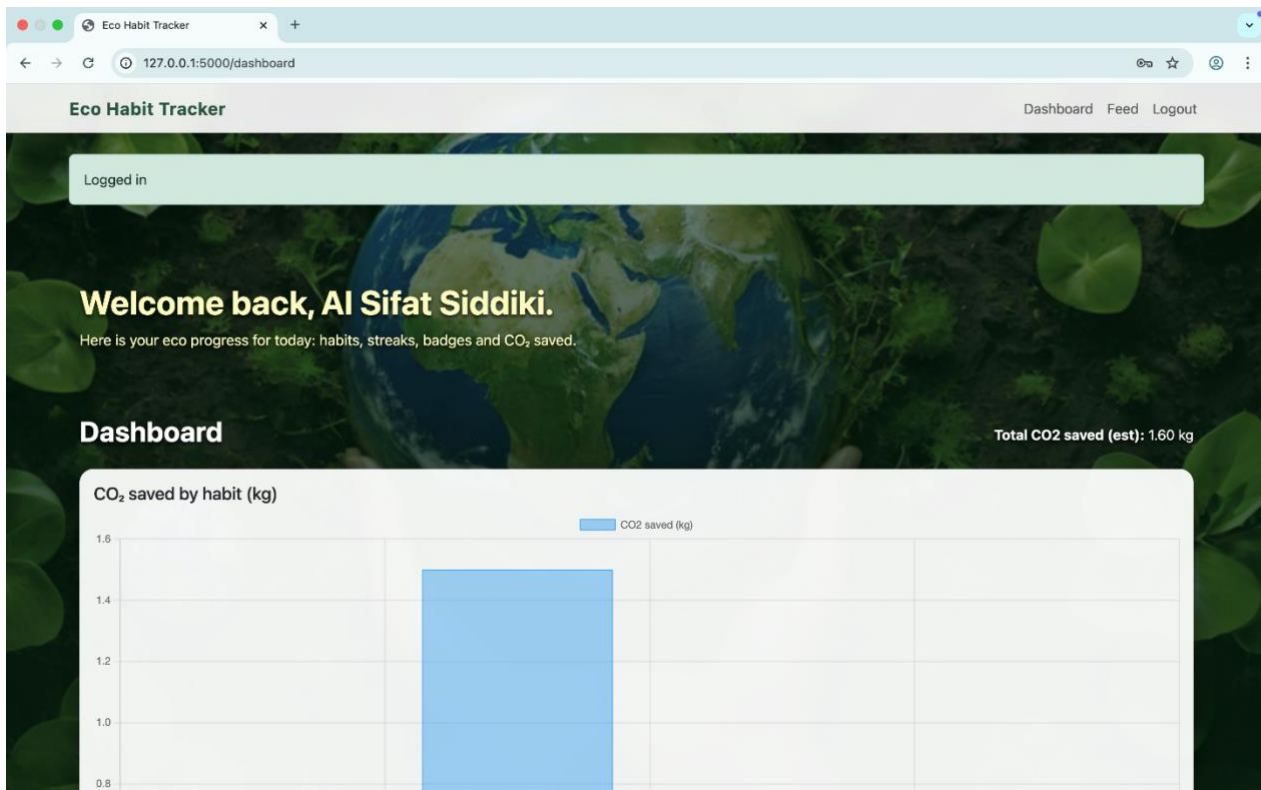
Planned: The Eco-Dashboard is the planned central hub of an application. It gives users a view into their current streaks, their overall environmental impact, and easy access to the community feed, badges, and their habit tools.

Implemented: In the final version, the dashboard provides:

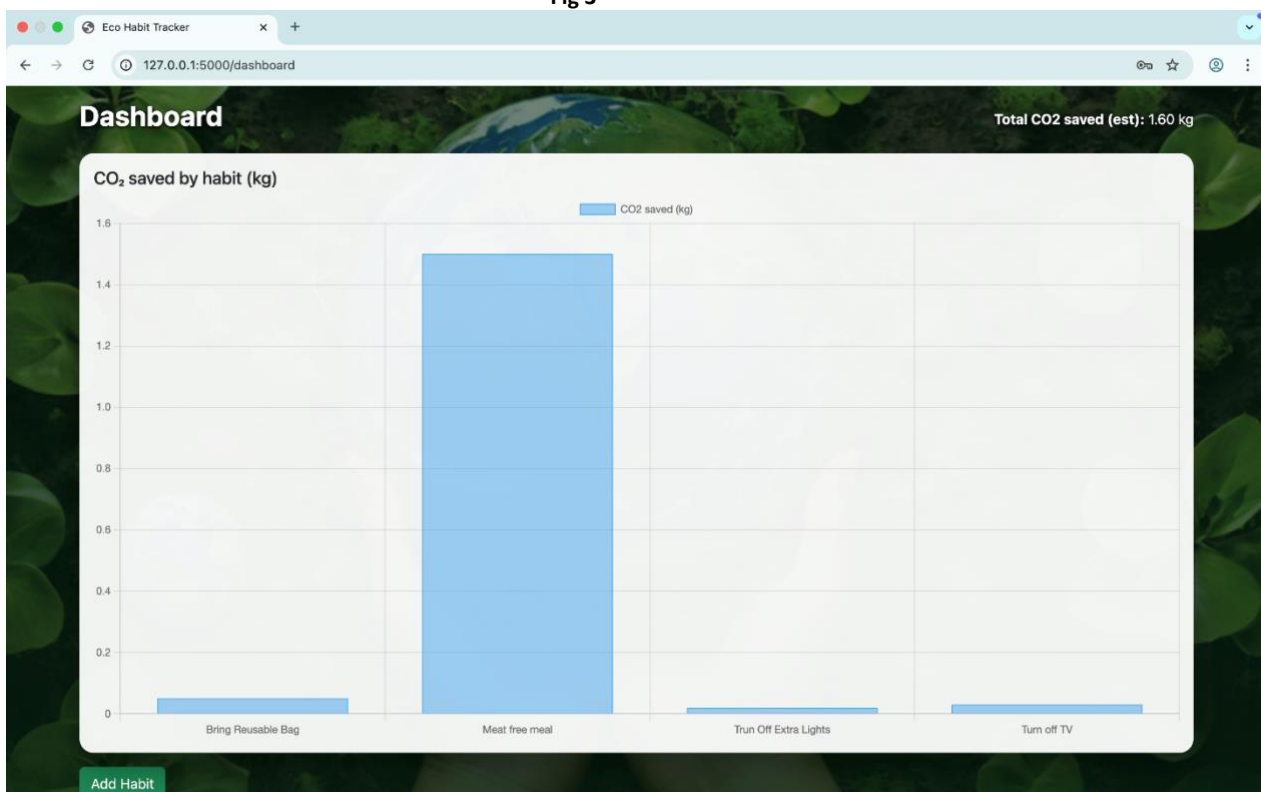
- A welcome panel that greets the user by name
- A bar chart showing CO₂ saved per habit (in kg), built from a small JSON endpoint that aggregates logs by habit
- A list of the user's habits, including a short description, CO₂ saved per action, and the current streak
- Buttons to add a new habit, open individual habit pages, and jump straight to the Community Feed.

The layout is intentionally clean so that streaks and CO₂ savings are easy to read at a glance without the page feeling crowded.

Differences: Some of the more ambitious analytics from Part 1 such as separate summaries for water, energy, or plastics did not make it into this version. I decided to only focus on CO₂ reductions as a single, clear metric. The dashboard stays fast, easy to read, and free from incomplete or distracting elements.



-Fig 3



-Fig 4

[Screenshot: Dashboard with bar chart, habit list, and CO₂ summary]

Habit management

Planned: The original feature list put a lot of weight on flexible habit management: users should be able to define eco habits that fit their own routines and see the CO₂ savings for each action.

Implemented: In the final system, habit management works as follows:

- An **Add Habit** form where users enter a title, a short description, and a CO₂ saving per action (for example, 0.08 kg for using a reusable bottle).
- A **habit detail** page that shows the description, current streak, progress towards the next badge threshold, and a list of recent logs.
- A **“Log today”** button that creates an entry for the current date and updates both streaks and CO₂ totals.

Each HabitLog stores the date and links back to the user and the habit, so the app can calculate streaks and total CO₂ for each habit without needing extra summary columns in the database.

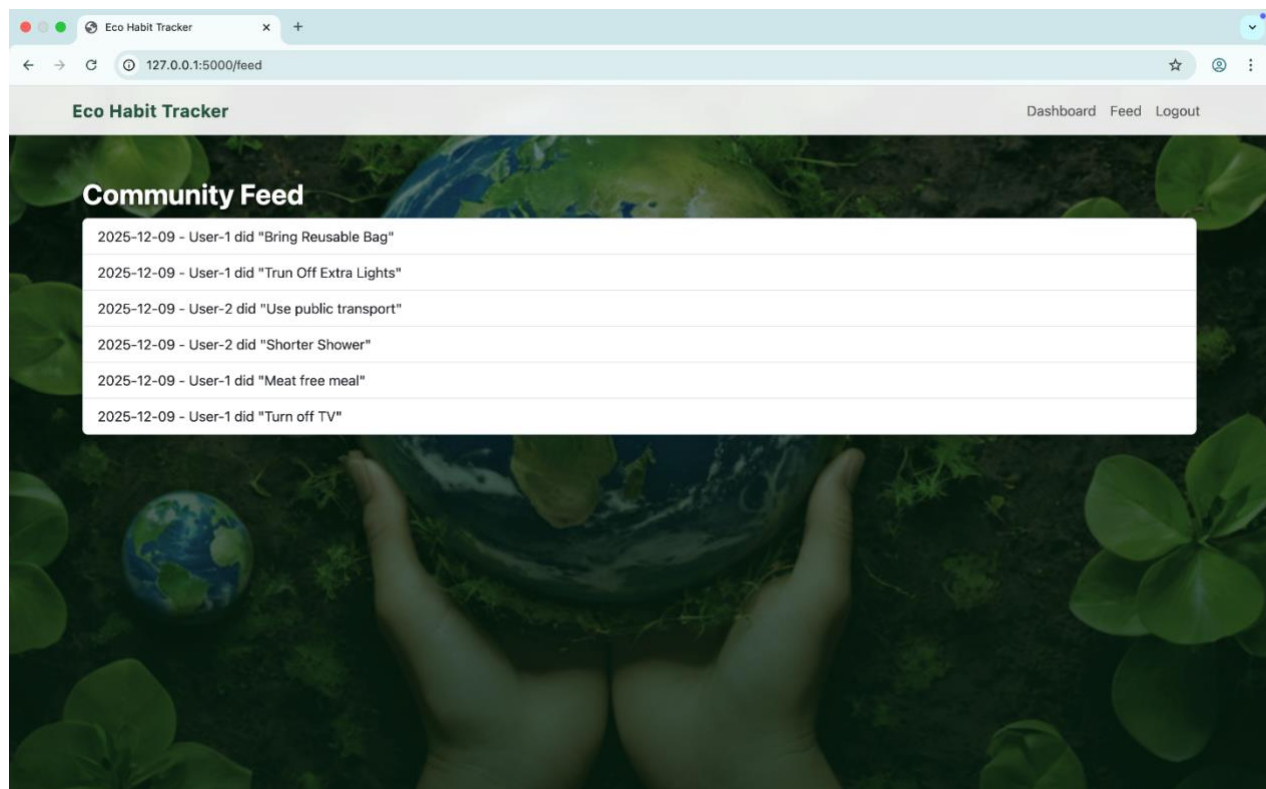
Differences: Right now you can create habits, but you can’t edit or delete them through the interface. That limitation keeps the streak calculations consistent and avoids a lot of messy edge cases.

Community Feed

Planned: In Part 1 I proposed a Community Impact Feed that would show anonymous group achievements. The aim was to encourage users by highlighting collective progress, without turning the app into a full social network.

Implemented: In the final system, the Feed page lists the most recent habit logs from all users as short messages, for example: Usernames are hidden and replaced with simple anonymised user IDs, which follows the privacy-focused design from Part 1. The feed is read-only and can be reached from both the top navigation bar and the dashboard.

Differences: At one stage I considered richer summaries such as “The community saved 3 kg of CO₂ this week” and filters by habit type. Because features like likes, comments, and following would bring extra moderation and privacy challenges, I intentionally kept this version simple. Even at its most basic, the feed clearly shows users that others are taking part in eco-friendly activities.



-Fig 5

[Screenshot: Community feed page listing recent eco actions]

Eco-badges

Planned: In Part 1, eco-badges were planned as a light form of gamification: nature-themed icons that users could unlock by maintaining streaks or reaching certain eco-impact milestones.

Implemented: In the final system, the badge feature works like this:

- A Badge table stores each badge along with a threshold value (for example, a minimum streak length).
- The application checks the user's streaks every time a habit is logged and creates a UserBadge record once a new badge is earned.
- The Badges section within the dashboard displays all of the badges that a user has obtained, as well as a brief description of each.

This means badges are actually tied to behaviour rather than just being decorative; they are awarded automatically and stored in the database.

Differences: At the moment, badges are shown as text rather than as illustrated icons, and there is no separate “My Eco-Badges” gallery page. The core logic is in place, but the visual design is fairly minimal. With more time, I would like to introduce custom icons and a dedicated badge view to make this part of the app feel more playful while still matching the overall eco theme.

Group challenges (not implemented)

Planned: In Part 1 I proposed group challenges, where small groups of users would work together towards shared goals, such as saving a certain amount of CO₂ within a month.

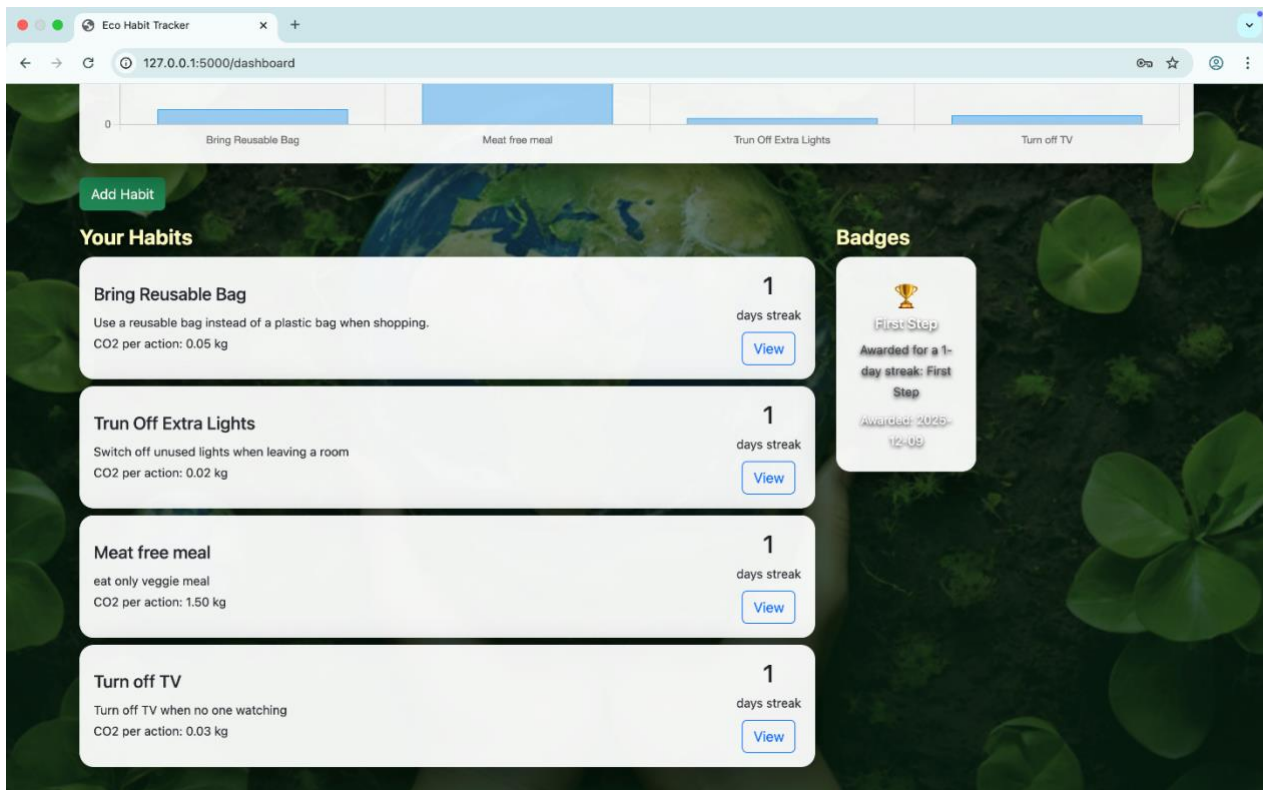
Implemented: Group challenges did not make it into the Part 2 implementation. To support them properly I would need additional tables (for groups, membership, and challenge participation), new pages for browsing and joining challenges, and server-side logic to combine progress across multiple users.

Differences: This is the main gap between the original design and the finished code. The positive side is that the current application stays coherent and does not include half-finished challenge features. The existing data model still makes it realistic to add group challenges in a future version by building on top of the current User and HabitLog tables.

2.3 Navigation and UI

The navigation I outlined in Part 1 is largely reflected in the running program. If a user chooses to register or log in from the landing page, they're taken straight to the main dashboard. From there, they have the ability to check their community feed, browse through their badges, add new habits, and review the details of the habits they already track.

This final layout ultimately was a bit more condensed than my original wireframes. I cut down some secondary links so the design works better on laptops and smaller screens. I also spent more time than expected on colour contrast and readability. The background image creates a strong eco atmosphere, but it initially made some text hard to read, so I darkened the overlay and used yellow or white for headings and form labels, often with a soft text shadow. This was not fully thought through in Part 1, but it turned out to be important for keeping the interface usable.



-Fig 6
[Screenshot: Dashboard navigation highlighting main links and layout]

2.4 Data model versus original ER diagram

The final data model stays very close to the ER diagram from the design report. The tables I implemented are:

- **User** – stores authentication details and a display name.
- **Habit** – holds the habit title, description, and co2_saving per action.
- **HabitLog** – links a user to a habit on a specific date and records the CO₂ saved for that entry.
- **Badge** – stores the badge name, description, and threshold value.
- **UserBadge** – a link table that records which user has earned which badge.

I deliberately chose not to store derived values, such as total CO₂ per user, directly in the database. Instead, the app calculates totals and streaks on the fly from the HabitLog records. This approach makes sense for a coursework-scale project with limited data. It keeps the schema simple and reduces the risk of conflicts when logs are updated.

3. Enhancements and future work

Although the current version meets the main aims of the coursework brief, there are still several practical and achievable ways to extend Eco Habit Tracker further.

3.1 Improved analytics

The dashboard could do more than show static counts and a single bar chart. With a few extra queries and some client-side charting, it could display:

- Weekly or monthly trend graphs for CO₂ savings
- Comparisons between habits, highlighting which ones contribute the most over time
- Simple summaries such as “CO₂ saved this week” or “Longest streak this month”

These additions would make progress easier to see at a glance, without changing the core interaction of logging daily actions.

3.2 Proper group challenges

Finishing the group challenge feature would be the biggest single extension. It would require:

- New tables for Challenge and ChallengeParticipant
- Pages for browsing available challenges, joining them, and tracking progress
- Clear rules for when a challenge is completed and how users are rewarded

The main design challenge would be keeping the interface calm and easy to read while adding these elements, so the app does not slide towards the more hectic, game-like style of Habitica.

3.3 Reminders and notifications

Right now, the app is completely passive nothing happens until the user decides to open it. In practice, people often forget. In the future, a mail service or background tasks could send gentle reminders or weekly summaries by email. To be useful rather than annoying, these reminders would need to be optional and carefully timed.

3.4 UX polishing and accessibility

The current interface works, but there are several small areas that could be improved:

- Clearer client-side validation messages on forms
- More consistent keyboard focus styles and ARIA labelling
- Stronger colour contrast for smaller text, such as badge descriptions

4. Challenges faced

Moving from a design document to a working Flask application revealed several challenges that were much less obvious on paper.

Authentication flow: It turned out that implementing authentication was more involved than just dropping in a login form. I needed to think about exactly what happens when an unauthorized user accesses a page marked as restricted, how they must be routed around, and where they are to be sent following their login. A few small errors led to some redirect loops and empty panels, which helped me to remember that one should always test things not only with regard to server logic but also from a user's perspective.

Streak logic: Streak calculation was another tricky area. My first idea was to store a single “streak” number on each habit, but that became unreliable as soon as I tried backdating logs or correcting errors. I eventually switched to calculating streaks from the ordered HabitLog dates for each habit, treating gaps carefully. The goal was to keep the rules fair and predictable without being unnecessarily strict.

Working with a real database: Unlike small classroom examples, this project used a database I genuinely did not want to break. Changing the schema often meant recreating tables, repopulating test data, and re-running seed scripts. That experience made me more cautious about altering models and more aware of how tightly application code, schema design, and real data are connected.

Template errors and debugging: Jinja2 is powerful but unforgiving when there is a typo. A single incorrect variable name in the habit detail template caused a TemplateSyntaxError that took the whole page down. Tracking it down forced me to get comfortable reading Flask tracebacks, isolating the problematic template, and testing changes in small steps instead of editing several files at once.

5. Achievements and learning

Even with a few compromises, I'm happy with how much of the original concept made it into the finished application. The core flow is clear and simple: users create an account, then create habits and log their daily actions, with the ability to check their progress on their dashboard. Badges and community feed add a light extra layer of motivation, without getting in the way of the main habit-tracking task.

In terms of learning, the project pushed me to turn abstract ideas into actual routes, templates, and database queries. I could not just write a feature list; I had to build each feature and then deal with the bugs and edge cases that showed up. At the end of this assignment, I feel more confident about the following:

- Structuring a small Flask project into sensible modules
- Thinking about relational data and many-to-many relationships
- Combining server-side rendering with a small amount of JavaScript for visualisation
- Iterating on the design based on what works on screen, not just what looks good in a wireframe

Overall, Eco Habit Tracker turned into a practical test of whether I could turn a written design into maintainable code within real time and scope limits. That experience should be useful both for future coursework and for any role where I'm expected to deliver working software, not just plans.

6. Technologies, libraries and installation

The application is built with **Python 3** and the **Flask** micro-framework, which I use for routing, templating, and session management. Flask suits this coursework well because it keeps me close to core web concepts while still offering enough abstraction to stay productive. Data is stored in a **SQLite** database via **SQLAlchemy**, which maps the ER diagram from Part 1 into Python models and keeps deployment straightforward there is no need to run a separate database server.

On the front end, the app relies on server-rendered HTML templates styled with custom CSS. A small amount of JavaScript, using the **Chart.js** library, drives the bar chart on the dashboard. This approach keeps the codebase easy to follow and avoids the extra complexity of a full client-side framework for what is ultimately a relatively small project. To set up the project on a new machine:

1. Create and activate a Python virtual environment.
2. Install dependencies with `pip install -r requirements.txt`.
3. Run the seed script to populate the database with example data.
4. Start the Flask development server with `python app.py` and visit `http://127.0.0.1:5000`.

These steps are also shown in the accompanying screencast, where I walk through the main pages and briefly explain how they connect back to the original design.

7. Conclusion

In the design phase I described Eco Habit Tracker as a way to make sustainable habits feel visible and worth celebrating, without drowning people in complexity. The final implementation does not include every idea from that original vision, but it does offer a coherent experience: users can register, track their habits, see their impact in terms of CO₂ savings, and get a sense that other people are taking action as well.

If I continue the project, my main priorities would be richer analytics, fully developed group challenges, and further accessibility improvements. Even without those extensions, the current version shows that the core concept works in practice and that the Part 1 design can be turned into a functioning Flask application that is understandable, extensible, and realistic for a single developer to build within one trimester.

References

- [1] EcoHero, "Sustainability Tracker." [Online]. Available: <https://ecohero-app.com>. [Accessed: Oct. 12, 2025].
- [2] Habitica, "Gamified Task Manager for Habit Building." [Online]. Available: <https://habitica.com>. [Accessed: Oct. 15, 2025].
- [3] Flask Documentation. [Online]. Available: <https://flask.palletsprojects.com>. [Accessed: Oct. 14, 2025].
- [4] SQLAlchemy, "SQLAlchemy 2.0 Documentation." [Online]. Available: <https://docs.sqlalchemy.org>. [Accessed: Nov. 20, 2025].
- [5] SQLite, "SQLite Documentation." [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: Nov. 20, 2025].
- [6] Chart.js, "Chart.js Documentation." [Online]. Available: <https://www.chartjs.org/docs/latest/>. [Accessed: Nov. 20, 2025].
- [7] Edinburgh Napier University, "SET09103 – Advanced Web Technologies Module Brief," School of Computing, Engineering & the Built Environment, 2025.
- [8] 40735842, "Eco Habit Tracker: Design Report (Part 1)," Edinburgh Napier University, 2025.